

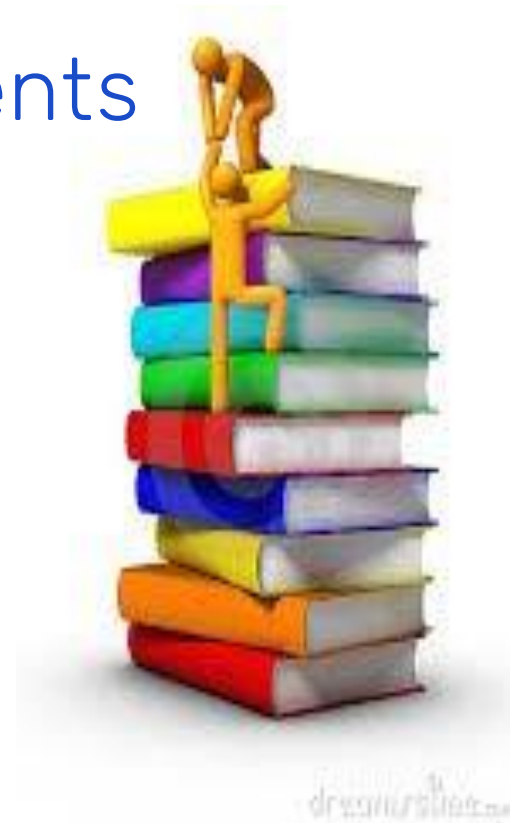
SOLVING PROBLEMS BY SEARCHING

PROBLEM FORMULATION

פרק 3
חלק 1

מה נלמד היום?

- Problem-solving agents
- Problem formulation
- Example problems
- Search algorithm



Problem-solving agents

- השלב הראשון בפתרון בעיה, הוא להגדיר באופן פורמאלי את הבעיה, מה המצבים שבה ומהן הפעולות האפשריות והמטרה הרצויה.
- הפתרון הוא תהליך ההחלטה אילו פעולות ומצבים יש לבחור כדי להגיע למטרה.

הגדרת הבעיה כדי לפתור אותה

עוזר בית

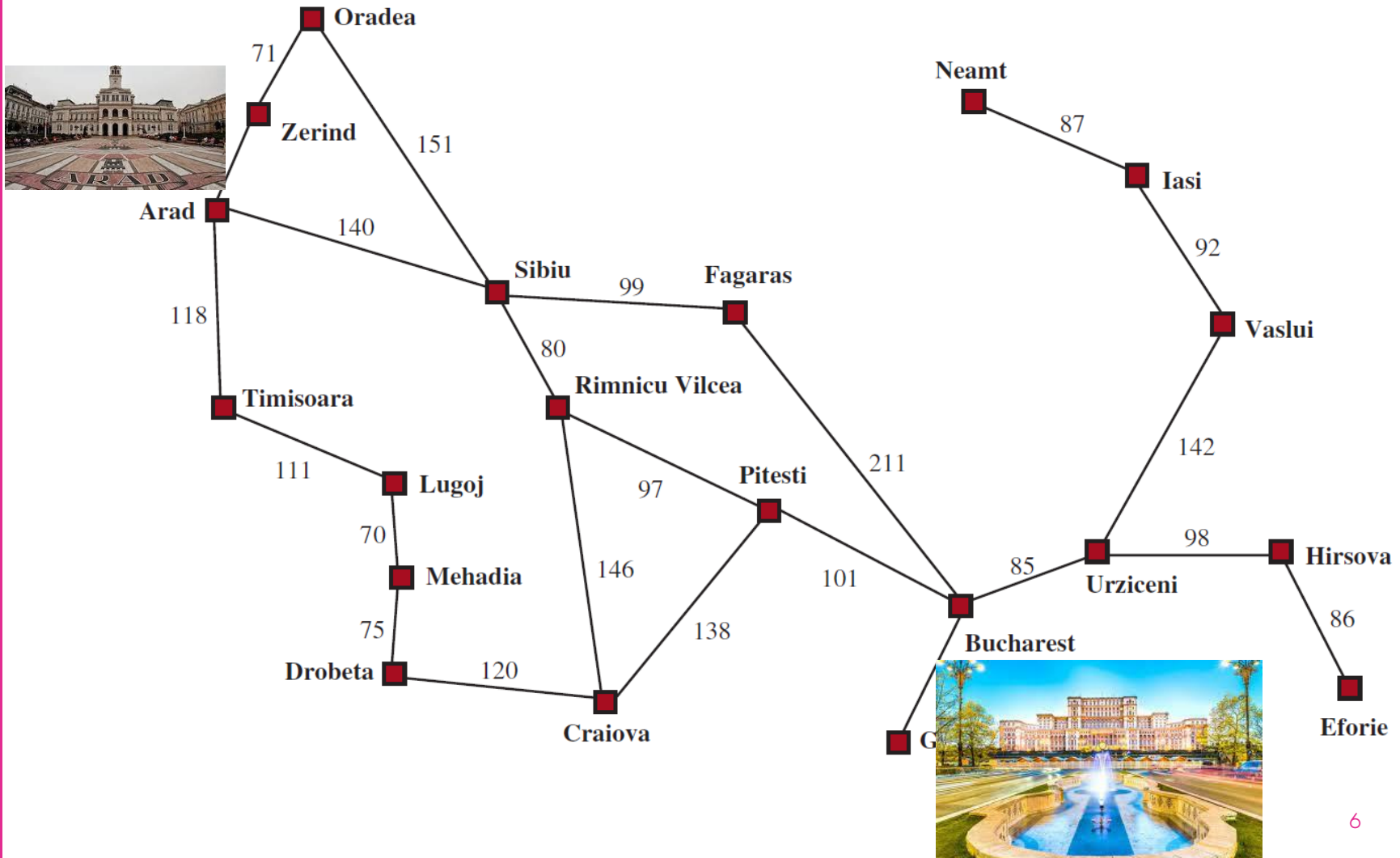
[https://www.youtube.com/watch?v=jglwgczi0frelod=](https://www.youtube.com/watch?v=jglwgczi0frelod=iaM&sp8)





טיול ברומניה

דוגמא: טיול ברומניה



סיווג סביבת בעיה

- סביבה גלויה או לא?
- סביבה סטאטית או דינאמית?
- סביבה אפיזודית או רציפה
- סביבה בדידה או מתמשכת?
- סביבה דטרמיניסטית או לא?
- סוכן יחיד או סוכנים מרובים?

סיווג סביבת בעיה

- סביבה גלויה
- סביבה סטאטית
- סביבה אפיזודית – ניתן גם לומר רציפה
- סביבה בדידה
- סביבה דטרמיניסטית
- סוכן יחיד

הגדרת בעיה

• מצב כללי של הבעיה

• מצב התחלתי

• **פעולות** - פונק' עוקב : עבור מצב ופעולה חוקית למצב

זה, מה יהיה המצב החדש?

• **מבחן המטרה** - האם הגענו למטרה?

• **עלות מסלול** - סכום עלות השלבים במסלול הפתרון.

דוגמא 1 : טיול ברומניה

מצב בעיה – הימצאות בעיר $city = X$

מצב התחלתי : הסוכן בעיר ארד. $city = Arad$

פעולות : מעבר מעיר לעיר שכנה. (**ביצוע פעולה חוקית**)

• למשל: $Successor((city = Arad), Go\ Zerind) = (city = Zerind)$

$Successor((city = city_i), Gocity_j) = (city = city_j)$

where $city_i, city_j$ are adjacent

מבחן המטרה : $city = Bucharest$?

עלות צעד : $stepcost = dist(city_i, city_j)$

פונקציית העוקב

פונקציית העוקב אינה המסלול לפתרון הבעיה!!

פונקציית העוקב מגדירה **מהם המצבים החוקיים לפי הגדרת הבעיה** שניתן לבצע במצב מסוים ומה התוצאות שלהם.

כדי להגדיר אותה:

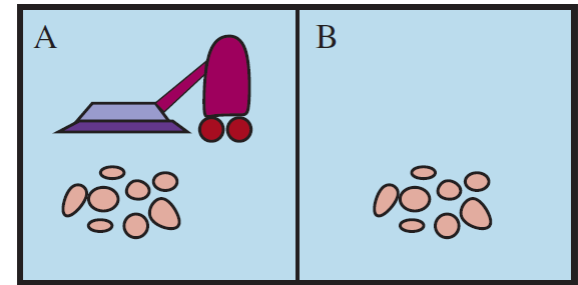
א. נגדיר בעברית מהן הפעולות החוקיות בבעיה לכל מצב.

ב. נכתוב דוגמא של עוקב ספציפי.

ג. נכתוב החוקיות באופן פורמלי.

ד. אם צריך, ניתן לכל פעולה תחום מתי היא חוקית.

Vacuum world :2 דוגמא



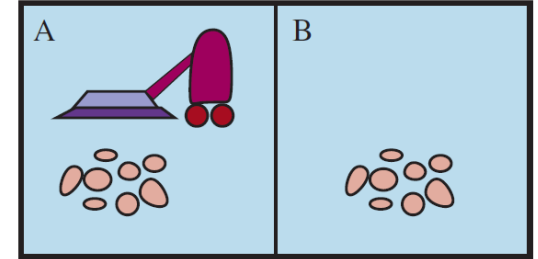
states?

successor?

goal test?

step cost?

Vacuum world :2 דוגמא



states? (A.dirty?, B.dirty?, location).

successor? $S(\text{state}, \text{Action}) = \text{new state}$

Example: $S((\text{True}, \text{True}, \text{A}), \text{Right}) = (\text{True}, \text{True}, \text{B})$

$S((\text{True}, \text{True}, \text{A}), \text{Left}) = (\text{True}, \text{True}, \text{A})$

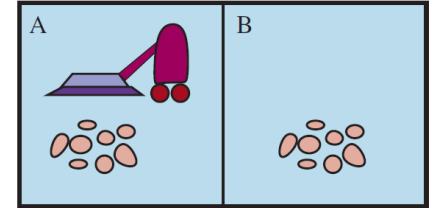
$S((\text{True}, \text{True}, \text{A}), \text{Suction}) = (\text{False}, \text{True}, \text{A})$

goal test? A. dirty =False and B.dirty =False

step cost? 1 per action

Vacuum world :2 אדגמ

Successor function example



Example: $S(\text{True, True, A}, \text{Right}) = (\text{True, True, B})$

$S(\text{True, True, A}, \text{Left}) = (\text{True, True, A})$

$S(\text{True, True, A}, \text{Suction}) = (\text{False, True, A})$

Successor function Rules

$S([X, Y, Z], \text{Right}) = (X, Y, B)$

$S([X, Y, Z], \text{Left}) = (X, Y, A)$

$S([X, Y, A], \text{Suction}) = (\text{False}, Y, A)$

$S([X, Y, B], \text{Suction}) = (X, \text{False}, B)$

The 8-puzzle Problem: 3 and 7

7	2	4
5		6
8	3	1

Start State

	1	2
3	4	5
6	7	8

Goal State

תרגיל : הגדירו את בעיית 8-puzzle באופן פורמאלי

7	2	4
5		6
8	3	1

Start State

	1	2
3	4	5
6	7	8

Goal State

states?

actions? Successor?

goal test?

step cost?

דוגמא 3: The 8-puzzle

- מצבים : מצב יתואר כמטריצה P של 3×3 .
- $P[i,j] = x$ עבור $x \in \{1,2,3,4,5,6,7,8, B\}$
- מבחן המטרה: הגענו למצב הרצוי כלומר
- $P[3, 1] = B, P[3,2] = 1, P[3,3] = 2, P[2,1] = 4 \dots$
- עלות מסלול – כל צעד עולה יחידה אחת.

פונקציית העוקב

פונקציית העוקב אינה המסלול לפתרון הבעיה!!

פונקציית העוקב מגדירה **מהם המצבים החוקיים לפי הגדרת הבעיה** שניתן לבצע במצב מסוים ומה התוצאות שלהם.

כדי להגדיר אותה:

א. נגדיר בעברית מהן הפעולות החוקיות בבעיה לכל מצב.

ב. נכתוב דוגמא של עוקב ספציפי.

ג. נכתוב החוקיות באופן פורמלי.

ד. אם צריך, ניתן לכל פעולה תחום מתי היא חוקית.

דוגמא 3: The 8-puzzle

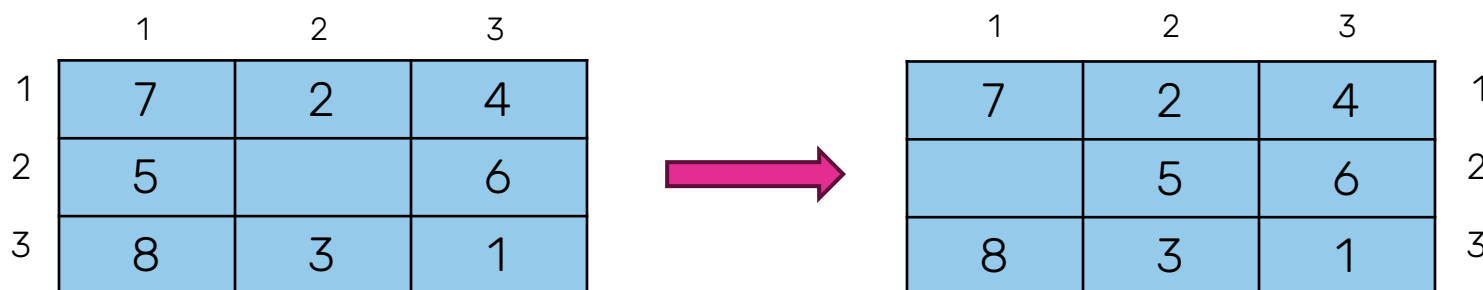
פונקציית העוקב – שלב 1 נגדיר בעברית מהן הפעולות החוקיות בבעיה לכל מצב.

- הזזת המרווח ע"י החלפה מקום אחד ימינה / שמאלה / למעלה / למטה, בתוך הלוח.

דוגמא 3: The 8-puzzle

• פונקציית העוקב – שלב 2 נכתוב דוגמא של עוקב

ספציפי.

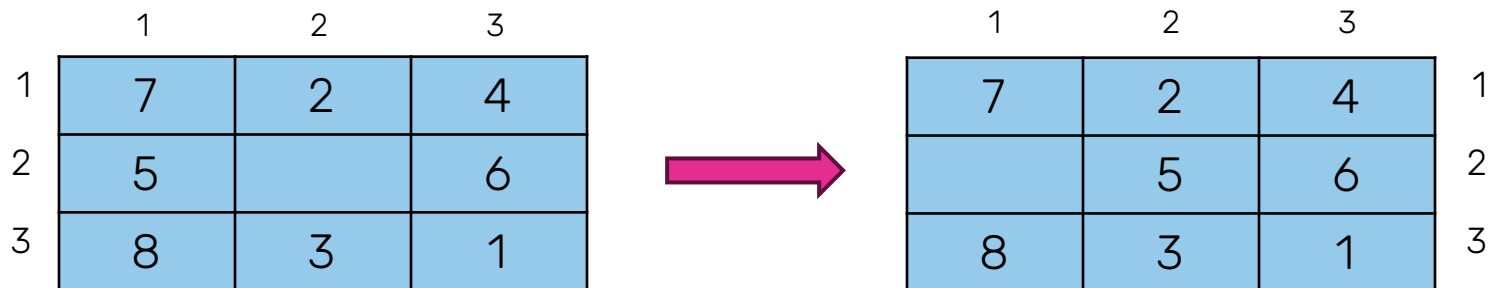


If $P[2, 2] = B$ then $S(\text{state}) =$ •

state after $\text{swap}(p[2, 2], p[2, 1])$ – left

דוגמא 3: The 8-puzzle

- פונקציית העוקב - שלב 3 נכתוב החוקיות באופן פורמלי



If $P[2, 2] = B$ then $S(\text{state}) =$

state after swap($p[2, 2]$, $p[2, 1]$) - left

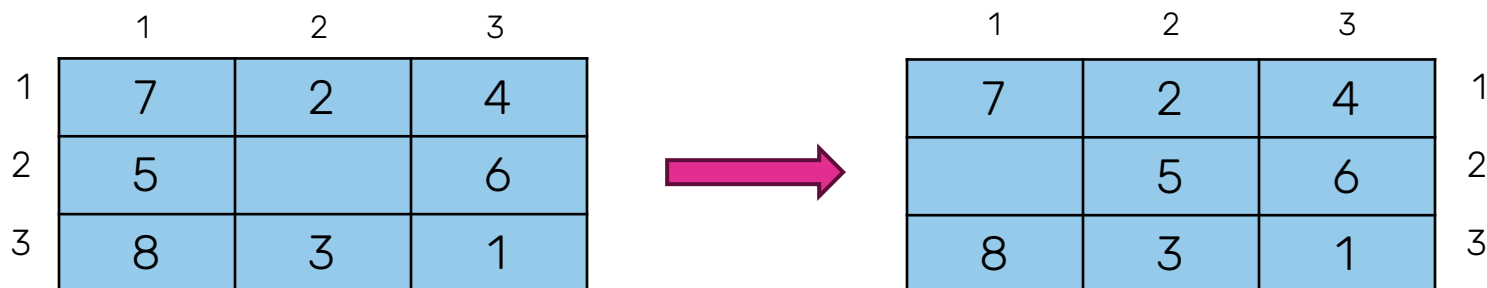
If $P[i, j] = B$ then $S(\text{state}) =$

- state after swap($p[i, j]$, $p[i, j-1]$) - left

דוגמא 3: The 8-puzzle

• פונקציית העוקב – שלב 4 אם צריך, ניתן לכל פעולה

תחום מתי היא חוקית.



If $P[2, 2] = B$ then $S(\text{state}) =$

state after $\text{swap}(p[2, 2], p[2, 1])$ - left

If $P[i, j] = B$ then $S(\text{state}) =$

state after $\text{swap}(p[i, j], p[i, j-1])$ if $j > 1$

דוגמא 3: The 8-puzzle

- פונקציית העוקב – הזזת המרווח ע"י החלפה מקום אחד ימינה / שמאלה / למעלה / למטה, בתוך הלוח.
- באופן פורמאלי (לא מלא):

If $P[i, j] = B$ then $S(\text{state}) =$

- state after swap($p[i, j]$, $p[i-1, j]$) - up
- state after swap($p[i, j]$, $p[i+1, j]$) - down
- state after swap($p[i, j]$, $p[i, j+1]$) - right
- state after swap($p[i, j]$, $p[i, j-1]$) - left

דוגמא 3: The 8-puzzle

- פונקציית העוקב – הזזת המרווח ע"י החלפה מקום אחד ימינה / שמאלה / למעלה / למטה, בגדר האפשרי.
- באופן פורמאלי מלא:

If $P[i, j] = B$ then $S(\text{state}) =$

- state after swap($p[i, j]$, $p[i-1, j]$) If $i > 1$
- state after swap($p[i, j]$, $p[i+1, j]$) If $i < 3$
- state after swap($p[i, j]$, $p[i, j+1]$) If $j < 3$
- state after swap($p[i, j]$, $p[i, j-1]$) If $j > 1$

דוגמא 4: בעיית Crossing the river

- בגדת נהר נמצאים איכר, זאב, כבשה וכרוב. כולם צריכים להגיע לגדת הנהר השנייה תוך שימוש בסירה שבה שני מקומות בהינתן הדרישות:
- אחד המקומות בסירה של הנהג יאויש תמיד בבן אדם.
- אסור להשאיר את הזאב והכבשה ביחד ללא השגחת האיכר על גדת נהר.



- אסור להשאיר את הכבשה והכרוב ביחד ללא השגחת האיכר על גדת נהר.



תרגיל כיתה: הגידרו באופן פורמאלי את בעיית Crossing the river

• בגדת נהר נמצאים איכר, זאב, כבשה וכרוב.
כולם צריכים להגיע לגדת הנהר השנייה תוך
בסירה שני מקומות כשאחד המקומות בסירה
יאוּש תמיד על ידי בן אדם.

states?

start state?

actions?

goal test?

step cost?

• אסור להשאיר את הזאב והכבשה ביחד ללא
השגחת האיכר על גדת נהר.
• אסור להשאיר את הכבשה והכרוב ביחד ללא
השגחת האיכר על גדת נהר.

Example: Crossing the river

• מצבים : וקטור שמתאר מי נמצא בגדה הראשונה
(Farmer, Sheep, Wolf, Cabbage)

מצב התחלתי – (F, S, W, C)

מצב סיום – (0, 0, 0, 0)

עלות מסלול – לכל פעולה עלות של יחידה אחת.

Example: Crossing the river

- פונקציית העוקב : העברה של הסירה מגדה לגדה לפי הכללים.

- באופן פורמלי :

- $S(F, S, W, C) = (0, 0, W, C)$
- $S(0, 0, W, C) = (F, S, W, C) \text{ OR } (F, 0, W, C)$
- $S(F, 0, W, C) = (0, 0, 0, C) \text{ OR } (0, 0, W, 0)$
- $S(0, 0, 0, C) = S(F, 0, W, C) \text{ OR } (F, S, 0, C) \text{ etc.....}$

Example: the missionaries and cannibals problem



- Three missionaries and three cannibals are on one side of a river that they wish to cross.
- A boat is available that can hold at most two people and at least one.
- You must never leave a group of missionaries outnumbered by cannibals on the same bank.
- Find an action sequence that brings everyone safely to the opposite bank.

Example: Complicated Crossing the river

• יש חידות שונות בסגנון.

להנאתכם [קישור](#)

<http://brainden.com/crossing-river.htm>

SOLVING PROBLEMS BY SEARCHING

Search algorithms

כשיש לסוכן מספר אפשרויות ללא עדיפות,
והוא מחפש רצף פעולות שיובילו למטרה,
הוא צריך אלגוריתם חיפוש.

Search algorithms

הרעיון המרכזי – עוברים על מרחב המצבים ע"י יצירת עוקב למצבים קיימים. מתחילים התהליך מהמצב ההתחלתי של הבעיה.

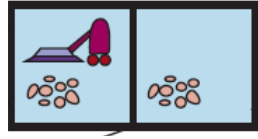
באופן זה בונים עץ (גרף) מעל מרחב המצבים כאשר:

קודקוד – מייצג מצב בעולם

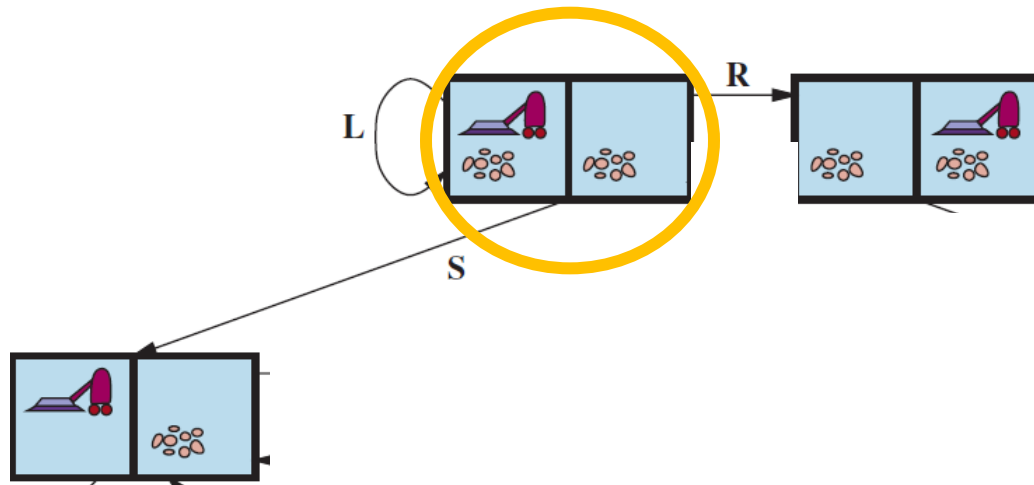
עלה – מצב שלא התפתחו ממנו מצבים נוספים

קשת – פעולה שמעבירה ממצב למצב הבא לפי פונקציית העוקב.

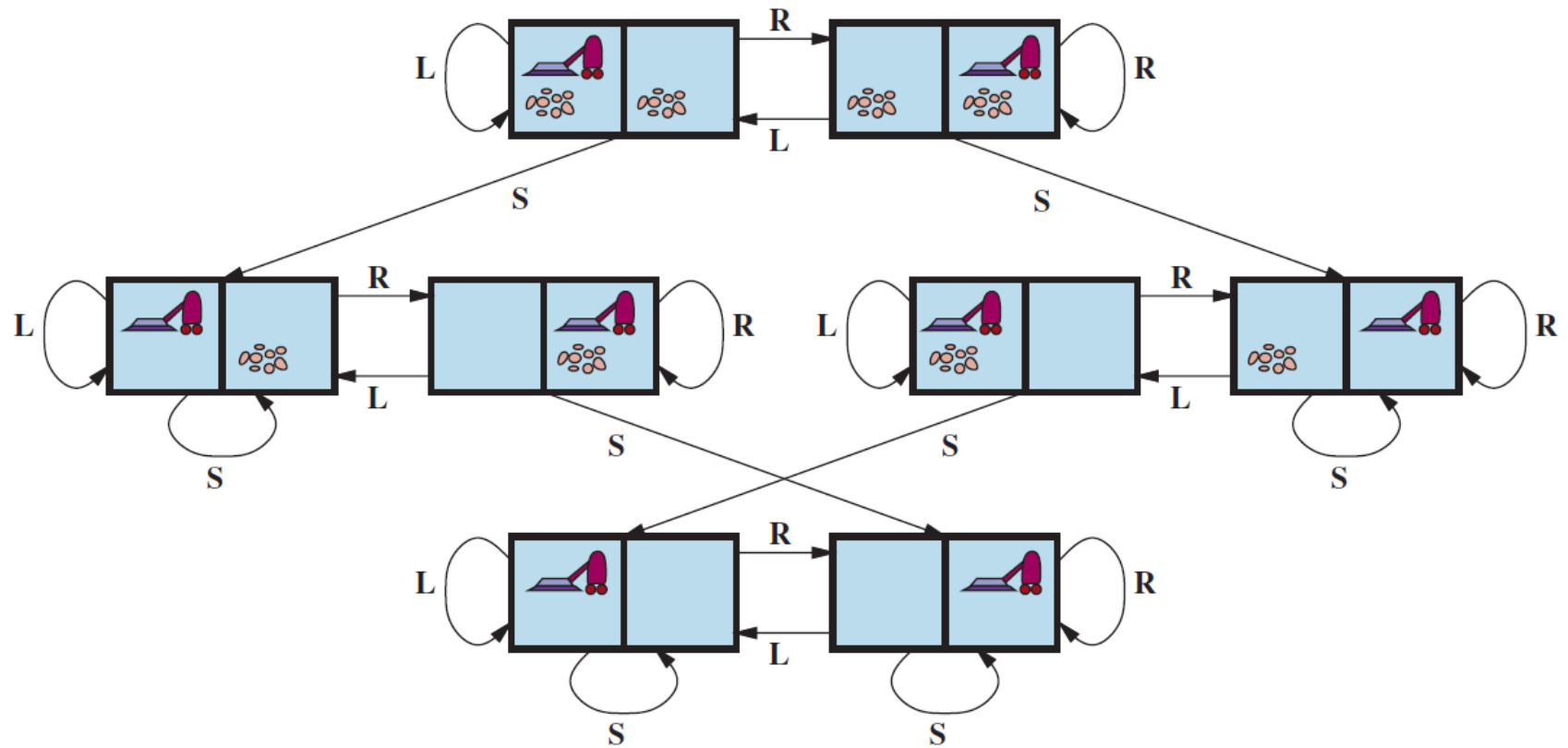
Vacuum world state space graph



Vacuum world state space graph

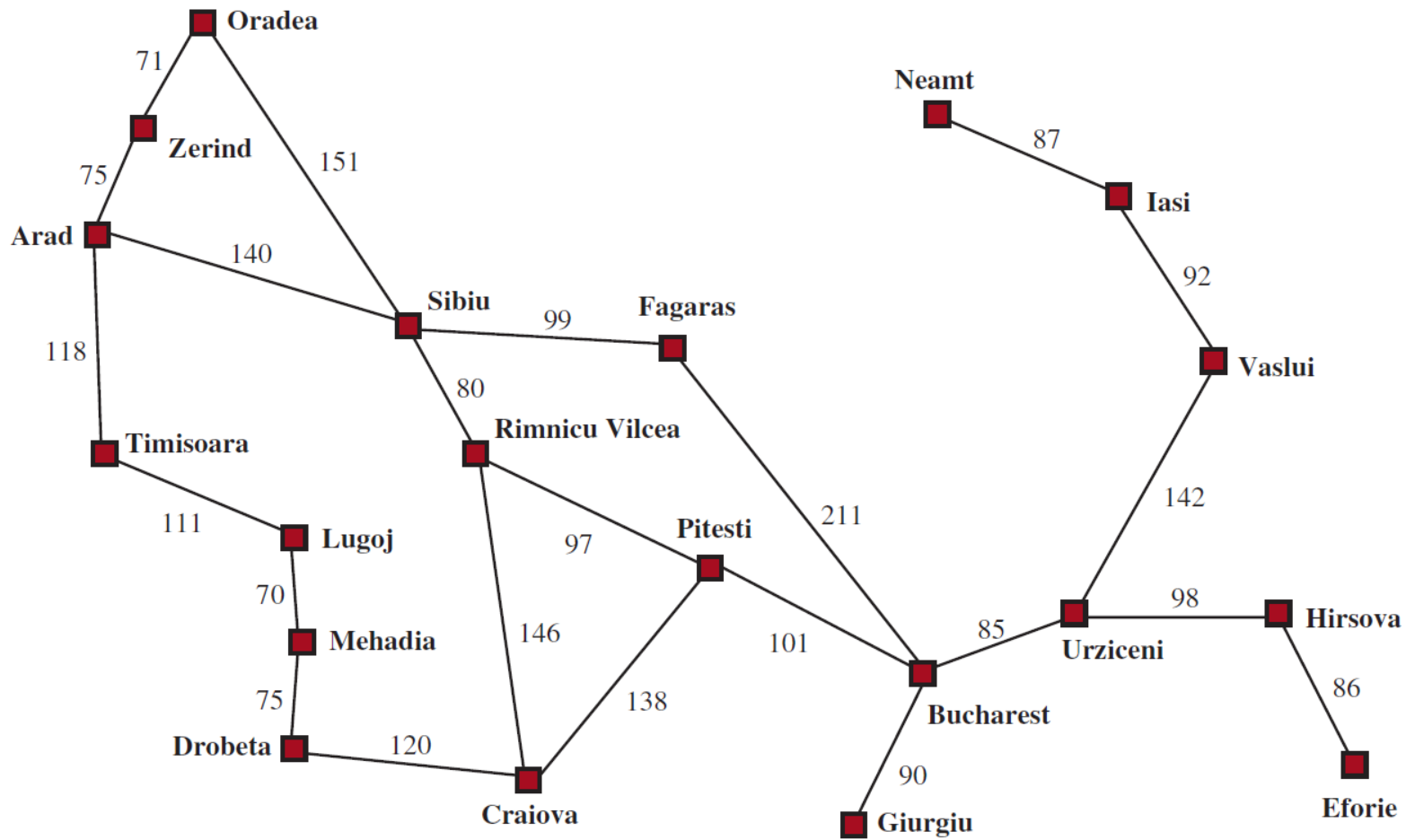


Vacuum world state space graph



Search algorithms Pseudo code

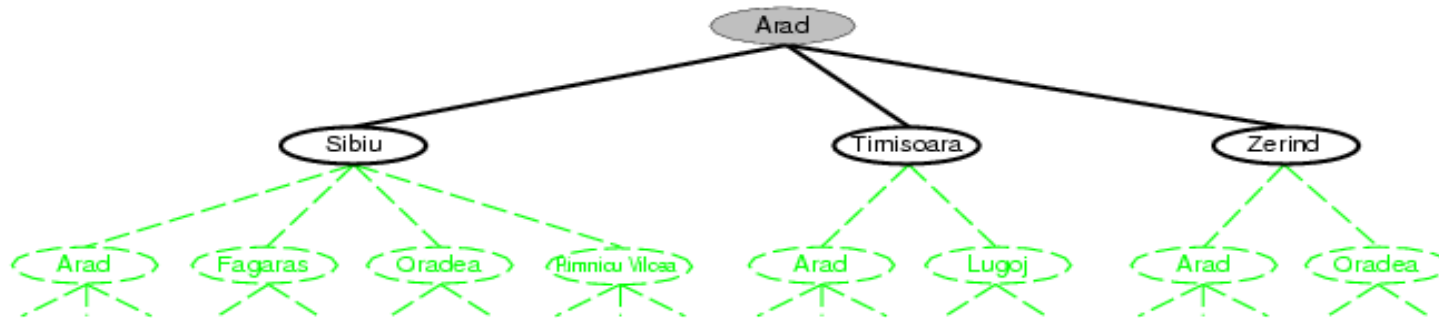
```
TreeSearch( problem, strategy) //returns a solution or failure
{
  Initialize the search tree by the initial state of problem.
  Loop do
    { if there are no leaves
      return failure
      Choose a leaf node according to strategy.
      if the node contains a goal state
        return the corresponding solution
      else
        { expand the node according to successor function
          add the resulting nodes to the search tree }
    }
}
```

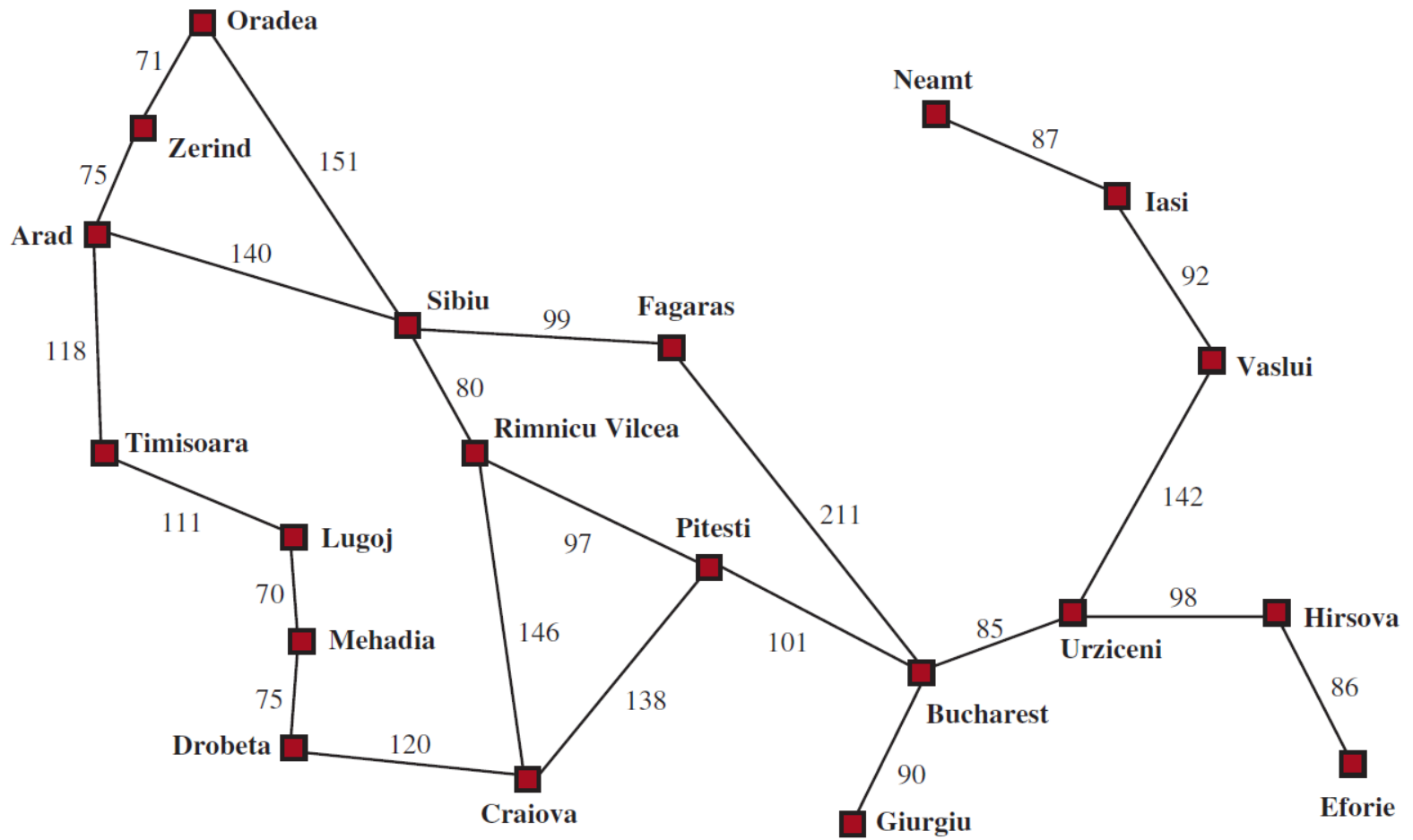


Tree search example

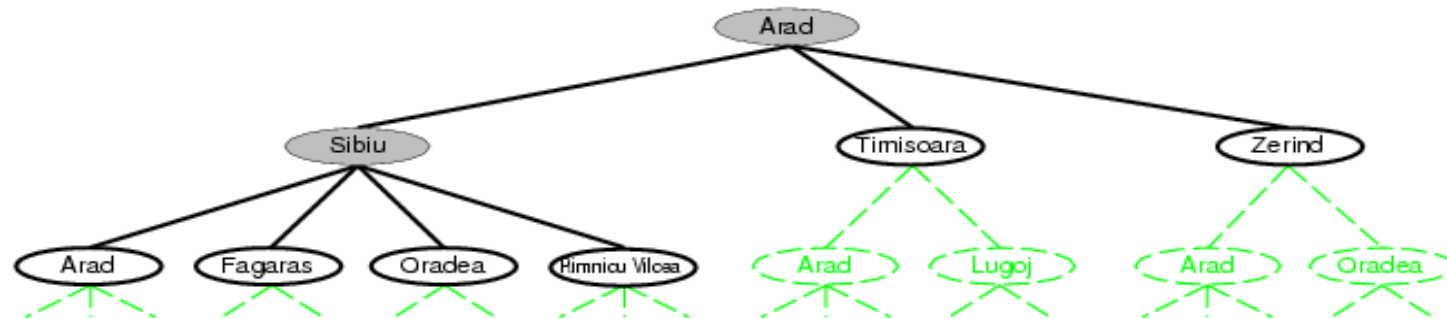


Tree search example



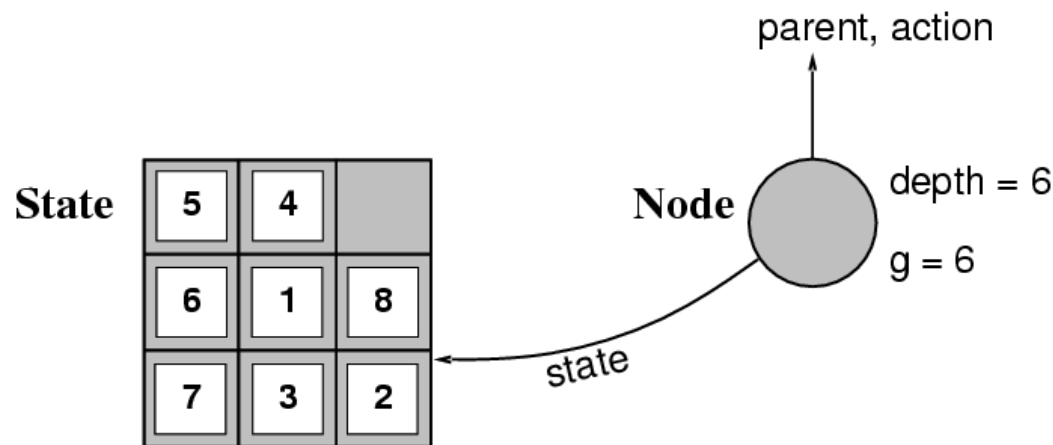


Tree search example

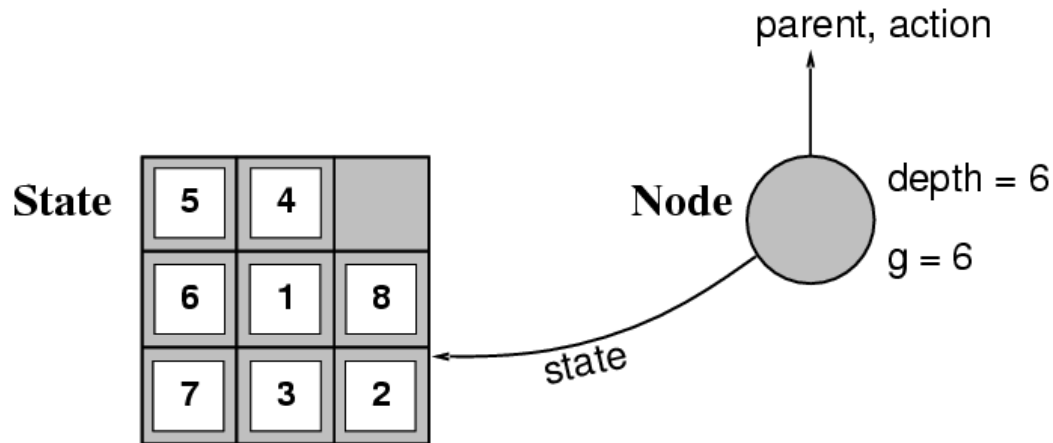


מצב לעומת קודקוד

- **מצב** – מתאר העולם כעת
- **קודקוד** – מבנה נתונים שהוא חלק מעץ החיפוש ובו:
 - state, parent node, action, path cost $g(x)$, depth



פיתוח קודקוד חדש



- The `Expand` function creates new nodes, using the `SuccessorFn` of the problem to create the corresponding states and filling in the various fields of the new nodes.

Implementation: general tree search

TreeSearch(problem, leaves)

{ leaves = Insert(InitialState(problem), leaves)

do

{if leaves is empty

return failure

node = RemoveFront(leaves)

if GoalTest(problem, node) = true

return solution(node)

leaves = InsertAll(Expand(node,problem), leaves)

}}

Implementation: general tree search

Expand(node , problem)

{ successors = empty set

 for each action, in SuccessorFn(node)

 { new = a new node

 State[new] = result of SuccessorFn(node)

 ParentNode(new) = node

 Actions[new] = action

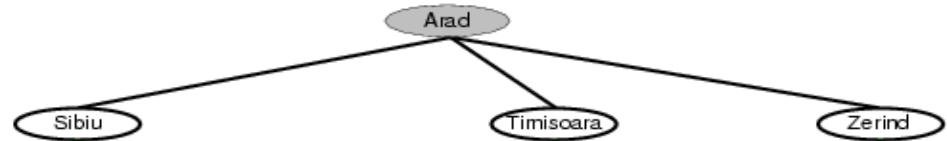
 PathCost[new]=PathCost[node]+StepCost(node, action, new)

 Depth [new] = Depth[node]+1

 Add new to successors }

return successors }

Implementation: general tree search



Expand(node , problem)

{ successors = empty set

for each action, in SuccessorFn(node)

{ new = a new node

State[new] = result of SuccessorFn(node)

ParentNode(new) = node

Actions[new] = action

PathCost[new]=PathCost[node]+StepCost(node,
action, new)

Depth [new] = Depth[node]+1

Add new to successors }

return successors }

Expand(Arad , problem)

{ successors = empty set

for each action, in SuccessorFn(node)

{ new = a new node

State[new] = Sibiu

ParentNode(new) = node(Arad)

Actions[new] = GoSibiu

PathCost[new]=0 + 140

Depth [new] = 0 + 1

Add new to successors }

return successors }

Search Strategies

• אסטרטגיית חיפוש מוגדרת ע"י קביעת **סדר פיתוח**
הקודקודים.

• הערכת אלג' חיפוש:

- **שלמות** – האם תמיד מחזיר פתרון כשקיים פתרון?
- **אופטימאליות** – האם מוצא פתרון אופטימאלי ביחס לעלות המסלול?
- **סיבוכיות זמן** – כמה זמן צורך האג'?
- **סיבוכיות מקום** – מספר הקוד' הנשמר בזיכרון

סיבוכיות זמן ומקום נמדדת ביחס למדדים:

- b : maximum branching factor of the search tree
- d : depth of the least-cost solution
- m : maximum depth of the state space
(may be ∞)